OS LAB REPORT

NAME : K SRI CHARAN REDDY
ROLL NO : 2401MC53
ASSIGNMENT NO : 09

1. Printing the Page Table of a Process

Kernel Modifications

- **System Call `vmprint()` (`vm.c`):**
  - Walks the two-level page table of the calling process (`myproc()->pgdir`) using `PDX(KERNBASE)` as the upper bound (`0x80000000`, 512 entries) to restrict traversal strictly to user space.
  - For each present page directory entry (`pgdir[i] & PTE_P`), increments the page-table page count and accesses the second-level page table (`pgtab`).
- **Syscall Infrastructure:** Added `SYS_vmprint` to `syscall.h`, `syscall.c`, `sysproc.c`, `user.h`, `usys.S`, and `defs.h`.

Output Explanation (`pttest`)

- **Initial State:** At startup, `pttest` has 3 valid pages:
  - `va 0x0`: Code/data segment (`[P: 1, W: 1, U: 1]`)
  - `va 0x1000`: Stack guard page (`[P: 1, W: 1, U: 0]`, user inaccessible)
  - `va 0x2000`: User stack (`[P: 1, W: 1, U: 1]`)
  - Page-table pages used: 1 (PDE 0).
- **Post-Allocation:** After calling `sbrk(4 * 4096)` and touching the allocated space, exactly four new user pages (`0x3000`, `0x4000`, `0x5000`, `0x6000`) appear with `[P: 1, W: 1, U: 1]`. Total valid pages increase from 3 to 7.

---

2. Handling Page Faults (Lazy Allocation)

Kernel Modifications

- **Lazy `sbrk()` (`sysproc.c`):**
  - Modified `sys_sbrk()` such that when `n > 0`, it increments `myproc()->sz` without calling `allocuvm()`.

    - Deallocation (`n < 0`) continues to invoke `growproc()` to release memory via `deallocuvm()`.
- **Fault Handling (`trap.c`):**
    - If `va < curproc->sz` and `va < KERNBASE`:
        - Checks that the page is not already present with protection violations (such as the stack guard page).
        - Allocates a physical frame with `kalloc()`, zeroes it with `memset()`, maps it with `mappages(..., PTE_W | PTE_U)`, and increments `curproc->pgfaults`.
- **Support in Core VM & Process Management:**
    - `vm.c`: Updated `deallocuvm()` and `copyuvm()` to skip unallocated lazy pages (`continue`) instead of triggering kernel panics.
    - `proc.h` & `proc.c`: Added `uint pgfaults` to `struct proc`, initialized in `allocproc()`.

Output Explanation (`faulttest`)

- Calls `sbrk(32 * 4096)`; initial faults remain 0 (proving allocation was delayed).
- Sequentially writes to 5 separate pages (pages 0, 4, 11, 22, 31). Each access triggers a page fault, dynamically allocating memory. `getfaults()` returns exactly 5.

---

3. Page Replacement Algorithms (FIFO and LRU)

User Program Implementation (`pagerepl.c`)

- **FIFO:** Tracks frame arrival times using a FIFO queue/timestamp. Upon a page fault with full frames, the oldest loaded frame is evicted.
- **LRU:** Maintains a timestamp of the most recent reference for each frame. When full, the frame with the smallest timestamp (least recently used) is evicted.
- **Execution & Formatting:** Simulates every access, printing per-step frame contents and hit/fault status, followed by a comparative summary table.

Results & Verification

Tested on reference string: `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1` (20 references):

Frames	FIFO Faults	FIFO Hit Ratio	LRU Faults	LRU Hit Ratio	Notes
3	**15**	**25.0%**	**12**	**40.0%**	Exactly matches assignment expected values
4	**10**	**50.0%**	**8**	**60.0%**	Both show increased hit ratio with 4 frames
5	**9**	**55.0%**	**7**	**65.0%**	LRU consistently outperforms FIFO

- **Observation:** LRU takes advantage of temporal locality by preserving recently referenced working set items (e.g., pages 0, 2, 3). FIFO replaces pages solely based on arrival age, leading to premature eviction of actively reused pages.